

NUNCHI · SECURITY REVIEW

Nunchi Migration Review.

A security review of Migration Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Nunchi	Migration Review	Guardian	November 24 – November 27, 2025	December 5, 2025

§ 00

Contents.

34 pages — overview, confidence ranking, scope, 20 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 20 findings	P. 09
§ 05	Disclaimer	P. 33
§ 06	About Guardian	P. 34

§ 01

Overview.

Guardian reviewed Migration Review across a single review round. The final report includes 5 High, 5 Medium, 6 Low, 4 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	Nunchi-trade/genesis-vaults
CLIENT	METHODOLOGY
Nunchi	Smart contract · Manual · Main Review
PROJECT	MAIN REVIEW COMMIT
Migration Review	137e7d5cec63
PREPARED BY	
Guardian	
AUDIT WINDOW	
November 24 – November 27, 2025	
FINAL REPORT	
December 5, 2025	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	5	0	0	0	5
Medium	5	0	1	0	4
Low	6	0	1	0	5
Informational	4	0	0	0	4
All severities	20	0	2	0	18

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Migration Review in `Nunchi-trade/genesis-vaults`. The scope includes 744 source lines across 6 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/migration/hub/GenesisVaultComposer.sol	97	198
src/migration/hub/NLPCLAIM.sol	256	442
src/migration/hub/VaultComposerSync.sol	165	349
src/migration/source/VaultMigrator.sol	183	358
src/migration/shared/libraries/MigrationErrors.sol	25	151
src/migration/shared/libraries/MigrationTypes.sol	18	49

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Unauthenticated Migration Processing	Validation	HIGH	RESOLVED	p. 9
H-02	Users cannot claim assets after a migration	Unexpected Behavior	HIGH	RESOLVED	p. 10
H-03	Insufficient swap slippage	MEV	HIGH	RESOLVED	p. 12
H-04	OFT Decimal Mismatch Causes Slippage Reverts	Logical Error	HIGH	RESOLVED	p. 13
H-05	SY Vault Users Can't Claim NLP Shares	Logical Error	HIGH	RESOLVED	p. 14
M-01	LZ Fee Underestimated	Logical Error	MEDIUM	RESOLVED	p. 15
M-02	Block confirmations are hardcoded	Unexpected Behavior	MEDIUM	RESOLVED	p. 16
M-03	Crosschain redeems for vault chains are refunded	DoS	MEDIUM	ACKNOWLEDGED	p. 17
M-04	Crosschain redeems can be grieved	DoS	MEDIUM	RESOLVED	p. 19
M-05	Withdrawal Delay Bypassed With Prefunded Address	Logical Error	MEDIUM	RESOLVED	p. 21
L-01	Native LayerZero Fee Refunds Can Become Stuck	Logical Error	LOW	RESOLVED	p. 22
L-02	Compose message options handling	Warning	LOW	RESOLVED	p. 23
L-03	Some accounts may lose access to migrated tokens	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 24

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-04	Migration fails if <code>_depositAndSend()</code> reverts	Warning	LOW	RESOLVED	p. 26
L-05	Unnecessary Approval For Native OFT Tokens	Logical Error	LOW	RESOLVED	p. 27
L-06	Missing Validation For <code>msg.value</code>	Validation	LOW	RESOLVED	p. 28
I-01	Missing Event Emission For Critical State Change	Events	INFORMATIONAL	RESOLVED	p. 29
I-02	Unnecessary Params In Verification Quote	Logical Error	INFORMATIONAL	RESOLVED	p. 30
I-03	Claim can be initiated for <code>address(0)</code>	Validation	INFORMATIONAL	RESOLVED	p. 31
I-04	State Variable Can Be Marked As Immutable	Gas Optimization	INFORMATIONAL	RESOLVED	p. 32

HIGH **H-01**

RESOLVED

Unauthenticated Migration Processing

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/migration/hub/GenesisVaultComposer.sol#L107-L176>

REVIEW Main Review

DESCRIPTION

Locally, the public `depositAndSend` only checks that `composeMsg` is non-empty; any EOA on the hub can call it with arbitrary `MigrationData`, mark a (vaultId, sourceEid) as processed, and register an allocation in `NLPCLaim`, blocking the real migration.

Cross-chain, `lzCompose` only verifies the sender is `ASSET_OFT/SHARE_OFT`; it never validates `composeFrom/srcEid` against a trusted migrator allowlist, so any source-chain user can craft an OFT send with fake `MigrationData` and achieve the same result.

This will result in a cheap, irreversible DoS/poisoning of migrations; attacker-controlled totalShares/nlpAmount skew claims

RECOMMENDATION

Restrict `depositAndSend` to trusted LayerZero compose flows; add (srcEid => trusted migrator) checks on `composeFrom/srcEid`; validate `MigrationData` against factory state or signatures before registering allocations.

PROOF OF CONCEPT <https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/guardian/test/guardian/pocs/ComposerCrossChainGriefPoC.t.sol>

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

HIGH **H-02**

RESOLVED

Users cannot claim assets after a migration

CATEGORY Unexpected Behavior
LOCATION NLPClaim.sol#L429-431
REVIEW Main Review

DESCRIPTION

Before claiming, `NLPClaim` uses `lzRead` to fetch the share balance of the user on the source chain. When the state is read and the result is confirmed by the DVN, `NLPClaim._lzReceive()` will be executed.

There, the following check is performed:

```
if (origin.srcEid != request.sourceEid) {  
    revert MigrationErrors.ChainMismatch(request.sourceEid, origin.srcEid);  
}
```

For `lzRead` the `origin.srcEid` is equal to the `READ_CHANNEL`. The app is set up in a way that the peer for `READ_CHANNEL` is the app itself.

```
function setReadChannel(uint32 _channelId, bool _active) public virtual onlyOwner {  
    _setPeer(_channelId, _active ? AddressCast.toBytes32(address(this)) : bytes32(0));  
}
```

In contrast, `request.sourceEid` will be equal to the chain the state is read from.

```
// Build and send read request  
bytes memory cmd = _buildReadCommand(vaultId, user, sourceEid);  
bytes memory options = this.combineOptions(READ_CHANNEL, CLAIM_MSG_TYPE, hex "");  
MessagingReceipt memory receipt = _lzSend(  
    READ_CHANNEL,  
    cmd,  
    options,  
    MessagingFee(msg.value, 0),  
    payable(msg.sender)  
);  
// Store request data for response routing  
claimRequests[receipt.guid] = ClaimRequest({  
    user: user,  
    vaultId: vaultId,
```

H-02 Users cannot claim assets after a migration

CONTINUED

```
sourceEid: sourceEid  
});
```

Because of this, `_lzReceive` will always revert and users won't be able to claim their assets. The tokens will have to be recovered by calling `NLPCLaim.recoverTokens()` and distributing them manually after that.

POC

A contract using `lzRead` was deployed on Arbitrum Sepolia. The contract emits the following event in its `_lzReceive()` function.

```
emit OriginSrcEidReceived(origin.srcEid);
```

The contract reads `WETH.decimals()` from Ethereum Sepolia. The [LZ message] (<https://testnet.layerzeroscan.com/tx/0xad7f975591dea81d365dd556f4b453ee28aceda315b2701b55cb84a6290cca48>) is successfully delivered. Exploring the transaction, it can be observed that the [event] (<https://sepolia.arbiscan.io/tx/0xc4bbc68c9171412837fed8d786c76584b6dbe01039e60c6dbb2064e15a5da896#eventlog#2>) emits the value `4294967295`, which is the read channel id.

RECOMMENDATION

Compare `origin.srcEid` against `READ_CHANNEL`, not `request.sourceEid`.

```
- if (origin.srcEid != request.sourceEid) {  
+ if (origin.srcEid != READ_CHANNEL) {  
- revert MigrationErrors.ChainMismatch(request.sourceEid, origin.srcEid);  
+ revert MigrationErrors.ChainMismatch(READ_CHANNEL, origin.srcEid);  
}
```

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

HIGH **H-03**

RESOLVED

Insufficient swap slippage

CATEGORY	MEV
LOCATION	VaultMigrator.sol#L303-304 https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/137e7d5cec63bbfbfebe897dc6606ce76aa0aae1/src/migration/source/VaultMigrator.sol#L303-L304
REVIEW	Main Review

DESCRIPTION

When a migration is executed through the `VaultMigrator`, all of the assets held by the vault are swapped to the desired stablecoin through curve.

```
uint256 expectedOut = curvePool.get_dy(fromIndex, toIndex, amountIn);
uint256 minAmountOut = (expectedOut * (10000 - maxSlippageBps)) / 10000;
// Approve Curve pool to spend input asset
IERC20(assetIn).forceApprove(address(curvePool), amountIn);
// Execute swap
amountOut = curvePool.exchange(fromIndex, toIndex, amountIn, minAmountOut);
```

The `maxSlippageBps` variable is applied to the result of `get_dy()` to determine the minimum amount of assets the contract is willing to accept. However, `get_dy()` returns a result based on the current chain state, so it will return the exact amount that will be received. Applying a discount to this value will always result in a smaller value than the tokens received, nullifying the slippage parameter. This means the swap is susceptible to being sandwiched in order to profit at the expense of the vault share holders.

RECOMMENDATION

Instead of applying the slippage to the result of `get_dy()`, consider having an exact value for `minAmountOut` passed when migrating.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

HIGH **H-04**

RESOLVED

OFT Decimal Mismatch Causes Slippage Reverts

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/migration/source/VaultMigrator.sol#L362
REVIEW	Main Review

DESCRIPTION

In `VaultMigrator._sendOFT` the migration uses `amountLD` and `minAmountLD` equal to the raw asset amount with no slippage buffer.

If the target stable's local decimals exceed the OFT's shared decimals (e.g., an 18-dec token like USD.e wrapped with an OFT set to 6 shared decimals), `OFTCore` will truncate "dust" when converting to shared units. The resulting amount credited on the destination is slightly less than `amountLD`, causing the send to revert against `minAmountLD` with error `SlippageExceeded`.

Therefore, any migrated value that is not a multiple of `10**((token decimals-shared decimals))` can thus stall migrations.

Similarly, when claiming NLP, the cross chain flow uses the same value for `amountLD` and `minAmountLD`. If the `SHARE_ERC20` decimals is greater than the `SHARE_OFT` shared decimals (6 by default), the transfer will revert due to slippage.

RECOMMENDATION

Before sending, account for shared/local decimal differences: set `minAmountLD` to 0 or to the OFT's `amountReceivedLD` from `quoteSend`. Add an explicit check of `token.decimals()` vs `IOFT.sharedDecimals()` and adjust slippage accordingly.

Keep in mind that this will leave some dust behind due to the truncation.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

HIGH **H-05**

RESOLVED

SY Vault Users Can't Claim NLP Shares

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/genesis/modules/VaultModule.sol#L225
REVIEW	Main Review

— DESCRIPTION

During claim verification, an `lzRead` is initiated to read the vault shares in the target chain. This read flow will call the `getUserSharesForClaim` function in the configured factory contract for given chain.

The current `VaultModule.getUserSharesForClaim` will return the share balance of users that deposited using the factory contract. Therefore, SY Vault users will have zero shares recorded in the factory, as the SY Vault itself is the share's owner.

Additionally, the SY Token vault mint ERC20 tokens to users. Even if `NLPclaim` can read the `burnedBalance` mapping in vault, user will only be able to claim once, as the `_hasClaimed` flag is set to true after claiming. In case any user receives more SY tokens, they won't be able to burn or claim, so they will need to transfer them to a different address.

— RECOMMENDATION

Consider creating a separate `initiateClaimSY` function that allows users to claim their shares from the SY Vaults, by reading the `vault.getUserSharesForClaim` which returns the user's burned balance.

Additionally, consider pausing SY transfers once the vault is migrated. Alternatively, document this scenario to users so they are aware of NLP claiming for SY holders.

PROOF OF CONCEPT <https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/guardian/test/guardian/pocs/InabilityToRedeemNlpPoC.t.sol>

— RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

MEDIUM **M-01**

RESOLVED

LZ Fee Underestimated

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/migration/source/VaultMigrator.sol#L275
REVIEW	Main Review

DESCRIPTION

The `quoteMigration()` encodes `composeMsg` using `MigrationData` passing it to `quoteSend`, while `executeVaultMigration` sends a much larger payload: `abi.encode(SendParam(innerParam with composeMsg=migrationData), uint256(0))`.

This underquotes the native fee, causing `executeVaultMigration` to revert at the send step if the operator uses the quoted fee.

RECOMMENDATION

Build the compose payload in `quoteMigration` exactly as in `executeVaultMigration`: wrap `MigrationData` inside an inner `SendParam` and then ABI-encode (`innerParam`, `minMsgValue`) for the outer compose. Then use that exact payload with `IOFT.quoteSend`

PROOF OF CONCEPT <https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/guardian/test/guardian/tests/GuardianTests.t.sol#L50>

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

MEDIUM **M-02**

RESOLVED

Block confirmations are hardcoded

CATEGORY Unexpected Behavior
LOCATION NLPclaim.sol#L465
REVIEW Main Review

DESCRIPTION

`NLPclaim._buildReadCommand()` hardcodes the block confirmations for the `lzRead` to 15 for all chains. While this configuration may work for some chains, it can be too low or too high for others.

For example, reading from a chain more susceptible to reorgs increases the risk of the `NLP` contract receiving state from a block that was removed.

If we take a look at the LayerZeroV1 [default configurations] (<https://docs.layerzero.network/v1/developers/evm/technical-reference/mainnet/default-config?>) for sending messages, we can see popular chains like `Optimism` and `Arbitrum` use 20.

In addition, if the oApp configuration changes in the future, it won't be possible to sync the `NLP` contract with the current approach.

RECOMMENDATION

Add a configurable mapping with block confirmations for each target chain and use that instead of hardcoding the blocks.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

MEDIUM **M-03**

ACKNOWLEDGED

Crosschain redeems for vault chains are refunded

CATEGORY DoS
LOCATION VaultComposerSync.sol#L355
REVIEW Main Review

DESCRIPTION

There is an `OFT` for the shares token of the vault on the hub chain. These ofts can be redeemed cross chain via the `lzCompose` flow. In case that `handleCompose()` fails, the tokens are refunded back to the sender on the source chain.

```
try
this.handleCompose{ value: msg.value }(_composeSender, composeFrom, composeMsg, amount)
{
emit Sent(_guid);
} catch (bytes memory _err) {
/// @dev A revert where the msg.value passed is lower than the min expected msg.value is
handled separately
/// This is because it is possible to re-trigger from the endpoint the compose operation with
the right msg.value
if (bytes4(_err) == InsufficientMsgValue.selector) {
assembly {
revert(add(32, _err), mload(_err))
}
}
_refund(_composeSender, _message, amount, tx.origin);
emit Refunded(_guid);
}
```

It's expected that `msg.value` is a positive value, otherwise the `_refund` call will fail since it calls `OFT.send()`. That `msg.value` is passed as `value` to `handleCompose()`.

Then the flow is `handleCompose() -> _redeemAndSend() -> _send()`.

M-03 Crosschain redeems for vault chains are refunded

CONTINUED

In the `_send()` function there is a check that requires `msg.value` to be 0 if the desired chain is the `VAULT_EID`. It's meant to protect users from sending value unexpectedly for same chain redeems, but in the case described, this check will cause the transaction to fail.

```
if (_sendParam.dstEid == VAULT_EID) {  
  /// @dev Can do this because _oft is validated before this function is called  
  address erc20 = _oft == ASSET_OFT ? ASSET_ERC20 : SHARE_ERC20;  
  if (msg.value > 0) revert NoMsgValueExpected();  
  IERC20(erc20).safeTransfer(_sendParam.to.bytes32ToAddress(), _sendParam.amountLD);  
}
```

Because of that, crosschain redeems where the destination chain is the hub will be refunded.

— RECOMMENDATION

You can move the `NoMsgValueExpected` check in the public `redeemAndSend` and `depositAndSend` functions or remove the `if(msg.value > 0) revert NoMsgValueExpected()` check and adding appropriate warnings to the frontend so users don't send funds when they don't have to.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

RESOLVED

Crosschain redeems can be grieved

CATEGORY DoS
LOCATION VaultComposerSync.sol#L168-172
REVIEW Main Review

DESCRIPTION

When `SHARE_OFT` is sent from a source chain to the hub chain, `lzCompose` will try calling `handleCompose()` and if it fails, the share tokens will be refunded back on the source chain.

```
try
this.handleCompose{ value: msg.value }(_composeSender, composeFrom, composeMsg, amount)
{
emit Sent(_guid);
} catch (bytes memory _err) {
/// @dev A revert where the msg.value passed is lower than the min expected msg.value is
handled separately
/// This is because it is possible to re-trigger from the endpoint the compose operation with
the right msg.value
if (bytes4(_err) == InsufficientMsgValue.selector) {
assembly {
revert(add(32, _err), mload(_err))
}
}
_refund(_composeSender, _message, amount, tx.origin);
emit Refunded(_guid);
}
```

Inside `_handleCompose()`, the `_redeemAndSend()` function will refund any surplus `msg.value` to `tx.origin`. This can be weaponized to grief the redeem:

- A user delegates code to their account with a type 4 transaction
- The user hijacks the call to `lzCompose()`
- The users sends more value than needed in order to trigger the refund
- The user reverts the first refund
- The `catch` block will be entered and the share asset will be sent back to the original owner on the source chain.

M-04 Crosschain redeems can be grieved

CONTINUED

In result the malicious user successfully grieved the redemption of the share assets at the expense of the send fee.

— RECOMMENDATION

Consider refunding to `address(this)` instead of `tx.origin` in `handleCompose()`. After `handleCompose()` finishes you can measure the change in the balance of the contract and if it's less than `msg.value`, refund the difference to `tx.origin`.

— RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

MEDIUM **M-05**

RESOLVED

Withdrawal Delay Bypassed With Prefunded Address

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/genesis/tokens/GenesisVaultSYToken.sol#L355
REVIEW	Main Review

DESCRIPTION

Cooldown is copied to recipients only when `balanceOf(to) == 0`. An attacker can pre-seed an address with a dust balance older than the delay, then transfer a large amount to that address; the recipient's `lastDepositTime` is not updated, letting them redeem immediately and bypass the intended withdrawal delay.

RECOMMENDATION

On every transfer propagate the max of sender/recipient cooldown (or use a balance-weighted accumulator), not just for zero-balance recipients, to prevent cooldown reset/bypass via pre-seeded addresses.

PROOF OF CONCEPT <https://gist.github.com/wafflemakr/1a30e5e712e2ed32da95649aa2a9fef3>

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

LOW **L-01**

RESOLVED

Native LayerZero Fee Refunds Can Become Stuck

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/migration/source/VaultMigrator.sol#L345
REVIEW	Main Review

DESCRIPTION

The `_send0FT` function hardcodes the refund address to `address(this)` and the contract exposes no method to withdraw native currency. Any overpayment of `msg.value`, LayerZero refund, or direct ETH transfers become trapped, leaving migration operators unable to recover funds.

RECOMMENDATION

Forward refunds to the caller/owner or add a payable withdraw function so native refunds are reclaimable.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

LOW **L-02**

RESOLVED

Compose message options handling

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

When a compose message is received by `GenesisVaultComposer`, the `msg.value` sent should be positive if the end chain is different than the hub and 0 otherwise.

If the OFT has enforced options for all composed messages to require `msg.value` sent, users would end up overpaying for the cases they don't need to send native tokens.

If there aren't enforced options, users should carefully pass their own extra options, increasing the risk of failures.

— RECOMMENDATION

Consider how should compose options be handled. If you decide to stick with enforced options, a refund mechanism for the unused native tokens may be helpful.

— RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

LOW **L-03**

ACKNOWLEDGED

Some accounts may lose access to migrated tokens

CATEGORY	Unexpected Behavior
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

During migration, all of the assets of the vaults are converted to a desired stablecoin and sent to the hub chain. Then each user can claim part of the shares of the vault on the hub chain. The amount they are eligible to claim depends on how much shares are they holding from the migrated vault. While this approach works for EOAs, it may fail for contracts. For example, a contract on chain A may not have the same address on chain B. This can happen for different reasons, one of which is the contract being deployed with the CREATE opcode after which the deployer's nonce is used to deploy another contract. In this scenario, the tokens of these contracts will be lost.

This can be mitigated if all of the contracts that don't have access to their address on the hub chain redeem their shares before the migration happens. This is possible because when a migration is initiated in `MigrationModule`, only deposits are disabled, but withdrawals can still happen until the delay of minimum one day passes and the migration is started. However, accounts holding less value than the minimum withdrawable amount will not be able to get their funds out because of the following check in

```
VaultModule

if (assets < vault.minWithdraw) {
  revert Errors.BelowMinimumWithdraw(vault.minWithdraw, assets);
}
```

Furthermore, if the same address is controlled by different entities on the two chains, one of them may claim the assets of the other.

— RECOMMENDATION

Before you initiate a migration, make it clear that such risks for contracts exists and set a long enough migration delay so users are able to successfully withdraw. Also consider skipping the `minWithdraw` check if the migration has already been initialized.

L-03 Some accounts may lose access to migrated tokens

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-04**

RESOLVED

Migration fails if `_depositAndSend()` reverts

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

During migrations, `VaultMigrator` sends tokens to the hub chain through the OFT and marks the vault as migrated `migrated[vaultId]`. Any further attempt to migrate the same vault will revert because it has already been migrated.

On the hub chain, the OFTs will be received and `handleCompose()` will be executed. If `handleCompose()` reverts, tokens will be refunded back to the source chain. This can happen for different reasons, one of which is a failure to deposit assets to the vault. In this case, the assets would be returned to the `VaultMigrator`, but the vault they correspond to will not be eligible for migration.

— RECOMMENDATION

Consider if the refund functionality is desired for migrations.

— RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

LOW **L-05**

RESOLVED

Unnecessary Approval For Native OFT Tokens

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/migration/source/VaultMigrator.sol#L328
REVIEW	Main Review

DESCRIPTION

The `VaultMigrator` will approve the stable asset to be spent by the OFT adapter. This will allow the `OFT.send` to pull assets from the migrator contract.

However, not all stable tokens use the `OFTAdapter` flow. For example, USD.e is an `OFT` token, which means `send` will actually burn tokens from the caller, and does not use `transferFrom`.

Therefore, the approval for certain tokens is unnecessary, and allowance will never be used.

RECOMMENDATION

Consider calling `OFT.approvalRequired` to determine if a `forceApprove` is needed for given token.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

LOW **L-06**

RESOLVED

Missing Validation For `msg.value`

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/migration/hub/NLPclaim.sol#L196>

REVIEW Main Review

DESCRIPTION

Some functions are marked as `payable` as it may involve a LZ fee to be paid in native currency.

The `NLPclaim.claim` function does not require any `msg.value` if `destEid == VAULT_EID`. However, it does not enforce zero value sent, no refund at the end, and no ownable function to recover native currency in the contract.

RECOMMENDATION

Consider adding the same validation as in composer when `destEid == VAULT_EID`:

```
if (msg.value > 0) revert NoMsgValueExpected();
```

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

INFORMATIONAL **I-01**

RESOLVED

Missing Event Emission For Critical State Change

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/migration/hub/NLPCLAIM.sol#L384
REVIEW	Main Review

DESCRIPTION

The `setVaultFactoryAddress` function updates critical configuration but does not emit an event, making it harder to track configuration changes off-chain.

RECOMMENDATION

Add an event `VaultFactoryAddressSet(uint32 indexed eid, address vaultFactoryAddress)` and emit it when the factory address is updated.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

INFORMATIONAL I-02

RESOLVED

Unnecessary Params In Verification Quote

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/migration/hub/NLPCLAIM.sol#L322
REVIEW	Main Review

DESCRIPTION

The `quoteVerification` uses `options` to calculate the quote, but the actual `initiateClaim` flow has empty options.

Additionally, this function also uses `msg.sender` as the user address, which is not ideal in view functions.

RECOMMENDATION

Consider removing the options param, and use empty bytes, just like in the `initiateClaim` flow, and instead add an `address` param for the user.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

INFORMATIONAL I-03

RESOLVED

Claim can be initiated for address (0)

CATEGORY	Validation
LOCATION	NLPCLAIM.sol
REVIEW	Main Review

DESCRIPTION

`NLPCLAIM.initiateClaim()` can be executed with `user == address(0)`. This will result in a message that always fails in the `_lzReceive` check due to the following constraint

```
if (request.user == address(0)) {  
  revert MigrationErrors.InvalidGuid(guid);  
}
```

RECOMMENDATION

Consider reverting if `initiateClaim` is called for `address(0)`

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

INFORMATIONAL **I-04**

RESOLVED

State Variable Can Be Marked As Immutable

CATEGORY	Gas Optimization
LOCATION	https://github.com/GuardianOrg/genesis-vaults-team1-1764005697720/blob/main/src/migration/source/VaultMigrator.sol#L44
REVIEW	Main Review

DESCRIPTION

The `VaultMigrator.destEid` state variable is only set during deployment, just like the `sourceEid` and `genesisFactory`. However, `destEid` is not marked as immutable.

RECOMMENDATION

Set `destEid` as an immutable param in `VaultMigrator`.

RESOLUTION

Status: Resolved.

<https://github.com/Nunchi-trade/genesis-vaults/pull/32>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.